Algoritmos de Acoplamento

CFD: Curso Básico

Professor: Adriano Possebon Rosa

Departamento de Engenharia Mecânica
Universidade de Brasília

Sumário

1 Algoritmos de Acoplamento Pressão-Velocidade

2 OpenFOAM
- fvSchemes
- fvSolution
- Exemplo

No caso de um escoamento incompressível, as equações governantes do problema são a equação da continuidade,

$$\nabla \cdot \mathbf{U} = 0 \ ,$$

e a equação de Navier-Stokes,

$$\frac{\partial \mathbf{U}}{\partial t} + \nabla \cdot (\mathbf{U}\mathbf{U}) - \nu \nabla^2 \mathbf{U} = -\nabla p \ .$$

Temos 4 equações e 4 incógnitas (as 3 componentes da velocidade e a pressão).

O problema é que não temos uma equação para a pressão.

Uma forma de contornar esse problema é por meio dos métodos de correção da pressão, que são os mais usados em CFD.

A ideia do método é criar uma equação para a pressão a partir da equação do momento, com a restrição da incompressibilidade.

Considere novamente a equação

$$\frac{\partial \mathbf{U}}{\partial t} + \nabla \cdot (\mathbf{U}\mathbf{U}) - \nu \nabla^2 \mathbf{U} = -\nabla p$$

Aplicando o divergente na equação, temos

$$\nabla \cdot \left(\frac{\partial \mathbf{U}}{\partial t} \right) + \nabla \cdot (\nabla \cdot (\mathbf{U}\mathbf{U})) - \nabla \cdot \left(\nu \nabla^2 \mathbf{U} \right) = -\nabla \cdot (\nabla p)$$

Considerando que o divergente da velocidade deve ser nulo, $\nabla \cdot \mathbf{U} = 0$, resulta

$$\cancel{\nabla \cdot \left(\frac{\partial \mathbf{U}}{\partial t} \right)} + \nabla \cdot (\nabla \cdot (\mathbf{U}\mathbf{U})) - \cancel{\nabla \cdot \left(\nu \nabla^2 \mathbf{U} \right)} = -\nabla \cdot (\nabla p)$$

Assim, temos uma equação para a pressão, dada por

$$\nabla^2 p = \nabla \cdot \mathbf{H}\left(\mathbf{U}\right) \ ,$$

em que

$$\mathbf{H}\left(\mathbf{U}\right) = -\nabla \cdot \left(\mathbf{U}\mathbf{U}\right) \ .$$

Dessa forma, a ideia desses métodos é resolver a pressão

$$\nabla^2 p = \nabla \cdot \mathbf{H}\left(\mathbf{U}\right)$$

e em seguida usar essa pressão para calcular uma nova velocidade

$$\frac{\partial \mathbf{U}}{\partial t} + \nabla \cdot \left(\mathbf{U}\mathbf{U}\right) - \nu \nabla^2 \mathbf{U} = -\nabla p \ .$$

Esse processo pode ter iterações internas, para melhorar/facilitar a convergência.

No OpenFOAM, a equação a ser resolvida é ligeiramente modificada.

Para deixar a equação para a pressão mais convergente, um termo $A\mathbf{U}$ (em que A representa os termos da diagonal da matriz dos coeficientes na equação do momento) é adicionado e subtraído da equação de Navier-Stokes:

$$\frac{\partial \mathbf{U}}{\partial t} + \nabla \cdot (\mathbf{U}\mathbf{U}) - \nu \nabla^2 \mathbf{U} - A\mathbf{U} + A\mathbf{U} = -\nabla p \ .$$

Seja agora

$$-\mathbf{H}(\mathbf{U}) = \frac{\partial \mathbf{U}}{\partial t} + \nabla \cdot (\mathbf{U}\mathbf{U}) - \nu \nabla^2 \mathbf{U} - A\mathbf{U}$$

Dividindo por A e aplicando o divergente, temos:

$$-\nabla \cdot \left(\frac{\mathbf{H}(\mathbf{U})}{A} \right) = -\nabla \cdot \left(\frac{1}{A} \nabla p \right) \ .$$

Assim, temos a equação para a pressão:

$$\nabla \cdot \left(\frac{1}{A} \nabla p \right) = \nabla \cdot \left(\frac{\mathbf{H}(\mathbf{U})}{A} \right) \ .$$

Calculada a pressão, a velocidade é obtida com a equação

$$\mathbf{U} = \frac{\mathbf{H}(\mathbf{U})}{A} - \frac{1}{A} \nabla p \ .$$

Temos, então, 3 passos básicos.

1) *momentum predictor* (opcional):

$$\frac{\partial \mathbf{U}}{\partial t} + \nabla \cdot (\mathbf{U}\mathbf{U}) - \nu \nabla^2 \mathbf{U} = -\nabla p$$

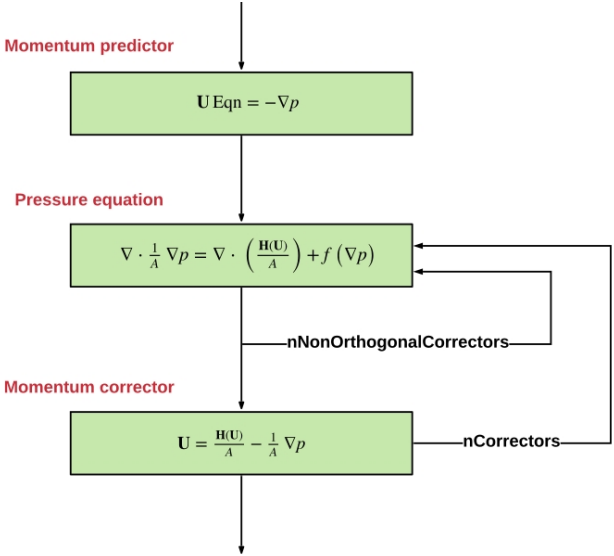
2) *pressure equation*:

$$\nabla \cdot \left(\frac{1}{A} \nabla p \right) = \nabla \cdot \left(\frac{\mathbf{H}(\mathbf{U})}{A} \right)$$

3) *momentum corrector*:

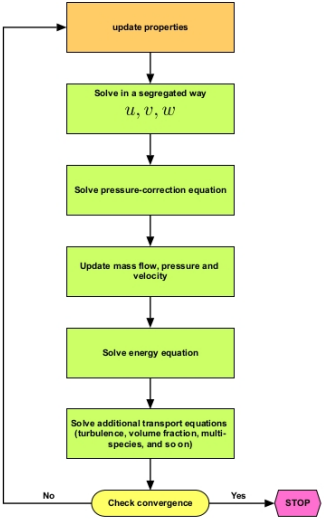
$$\mathbf{U} = \frac{\mathbf{H}(\mathbf{U})}{A} - \frac{1}{A} \nabla p$$

Esses são os passos básicos dos métodos de correção da pressão.



Os métodos de correção da pressão mais utilizados são:

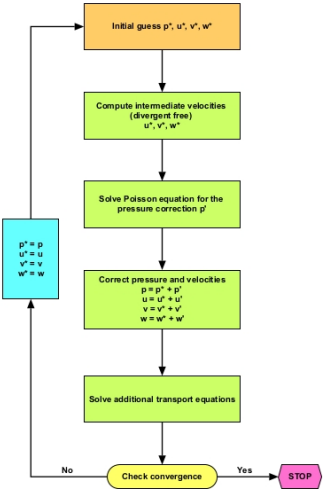
- **SIMPLE** (Semi-Implicit Method for Pressure-Linked Equations)
- **SIMPLEC** (SIMPLE Corrected/Consistent)
- **PISO** (Pressure Implicit with Splitting Operators)
- **PIMPLE** (PISO + SIMPLE)



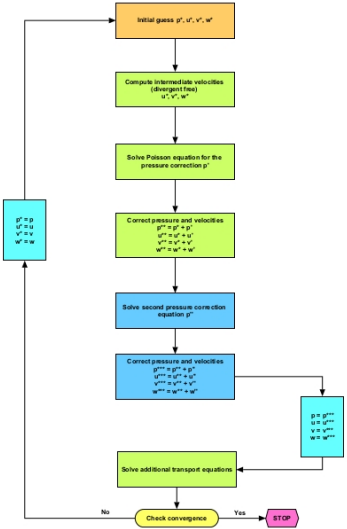
Os métodos SIMPLE e SIMPLEC são usados em simulações em regime permanente, enquanto os métodos PISO e PIMPLE são usados em simulações transientes.

Os métodos SIMPLE e PISO são apresentados no próximo slide.

Pressure-velocity coupling using the SIMPLE method



Pressure-velocity coupling using the PISO method



Sumário

1 Algoritmos de Acoplamento Pressão-Velocidade

2 OpenFOAM
- fvSchemes
- fvSolution
- Exemplo

Um caso típico no OpenFOAM tem 3 pastas:

- *0*
- *constant*
- *system*

Na pasta *0* nós entramos com as condições iniciais e de contorno do problema.

Na pasta *constant* nós temos as propriedades físicas do fluido, no arquivo *physicalProperties*, e as informações sobre a modelagem do transporte de momento, no arquivo *momentumTransport*.

Na pasta *system*, temos as informações sobre a geometria e a malha, no arquivo *blockMeshDict*, e sobre as configurações gerais da simulação, no arquivo *controlDict*.

Temos, ainda na pasta *system*, os arquivos ***fvSchemes*** e ***fvSolution***.

No arquivo **fvSchemes**, nós especificamos os esquemas de discretização usados no método de volumes finitos empregado para resolver as equações governantes.

As especificações sobre os métodos de resolução dos sistemas de equações lineares e sobre o algoritmo de acoplamento pressão-velocidade, por sua vez, são informadas no arquivo **fvSolution**.

Vamos analisar esses dois arquivos, começando com o fvSchemes.

Sumário

1 Algoritmos de Acoplamento Pressão-Velocidade

2 OpenFOAM
- fvSchemes
- fvSolution
- Exemplo

No dicionário ***fvSchemes***, nós vamos especificar os esquemas de discretização utilizados nas aproximações do método de volumes finitos.

Assim, precisaremos definir os seguintes subdicionários:

- ***ddtSchemes***
- ***gradSchemes***
- ***divSchemes***
- ***laplacianSchemes***
- ***interpolationSchemes***
- ***snGradSchemes***

Observação: o material desta seção foi retirado de *"Introduction to the FVM method: Standard practices in general CFD with applications to OpenFOAM"*, da Wolf Dynamics. Recomendo muito a leitura.

ddtSchemes se refere à forma como a derivada no tempo é discretizada.

As principais opções são:

- *steadyState*: para simulações em regime permanente.
- *Euler*: para simulações em regime transiente, primeira ordem.
- *backward*: para simulações em regime transiente, segunda ordem.
- *CrankNicolson*: para simulações em regime transiente, entre primeira e segunda ordem.

No caso transiente, *Euler* é o mais robusto, mas é de primeira ordem. *Crank-Nicolson 1* é o método de Crank-Nicolson puro (segunda ordem, mas pode apresentar oscilações indesejadas), enquanto *CrankNicolson 0* recupera o método de *Euler*. A opção *CrankNicolson 0.7* é uma boa escolha em várias situações, já que é estável e preciso.

```
ddtSchemes
{
   default    CrankNicolson 0.7;
}
```

No ***divSchemes*** nós definimos como são calculados os termos advectivos, ou seja, os termos do tipo $\nabla \cdot (\mathbf{U}\phi)$.

As principais opções são:

- *upwind*: primeira ordem, limitado.
- *linear*: central, segunda ordem, ilimitado.
- *linearUpwind*: segunda ordem, ilimitado.
- *vanLeer*: segunda ordem, TVD, limitado.
- *Minmod*: segunda ordem, TVD, limitado.
- *limitedLinear*: segunda ordem, ilimitado.

O método *upwind* é estável e robusto, mas é de primeira ordem e é muito difusivo. Os métodos de segunda ordem possuem uma precisão maior, mas podem se tornar oscilatórios. O ideal é um método de segunda ordem estável. Podemos começar a simulação com o método de primeira ordem e, depois de ver que tudo está funcionando bem, passar para um de segunda ordem. O melhor método vai depender do problema.

O tratamento dos termos advectivos é um dos maiores desafios em CFD.

```
ddtSchemes
{
   default       none;
   div(phi,U)    Gauss linearUpwind grad(U);
}
```

O dicionário ***gradSchemes*** indica a forma como o gradiente é calculado ($\nabla(p)$ e $\nabla(\mathbf{U})$, por exemplo).

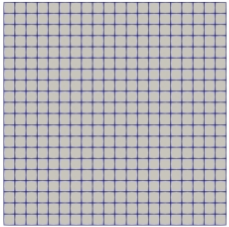
A opção mais usada é a *Gauss linear*. A opção *cellLimited Gauss linear 1* também é usada, principalmente em simulações com malhas não tão boas. A opção *cellLimited* limita o valor do gradiente de acordo com os valores da variável nas células mais próximas, aumentando a estabilidade da simulação.

```
gradSchemes
{
   default       Gauss linear;
   grad(U)       cellLimited Gauss linear 1;
}
```

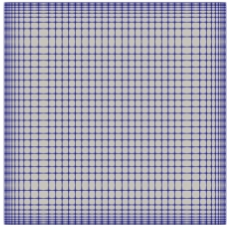
O dicionário ***laplacianSchemes*** está relacionado aos termos difusivos das equações, da forma $\nabla \cdot (\Gamma \nabla \phi)$.

As principais opções aqui são:

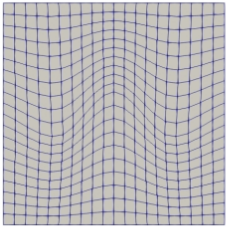
- *orthogonal*: para malhas perfeitas (ortogonal e com células idênticas), segunda ordem.
- *corrected*: para malhas não ortogonais e com diferentes tamanhos de célula.
- *limited*: para malhas não ortogonais e com diferentes tamanhos de célula.



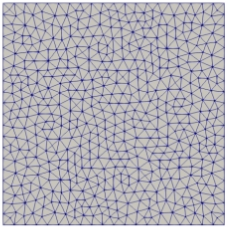
Perfect orthogonal mesh with no strectching
laplacianSchemes → orthogonal



Orthogonal mesh with strectching
laplacianSchemes → limited 1 or **corrected**



Mesh with some degree of non-orthogonality (low to medium)
laplacianSchemes → limited 1 to **limited 0.5**



General unstructured meshes
laplacianSchemes → limited 0.5

Para malhas com não ortogonalidade menor que 70°, você pode deixar o fator *limited* como sendo 1.

Para malhas com não ortogonalidade entre 70° e 85°, defina o fator como 0.5.

Para malhas com não ortogonalidade maior que 85°, o melhor é tentar fazer uma nova malha. Mas caso você queira usar essa malha, defina o fator entre 0.333 e 0.5.

```
laplacianSchemes
{
   default       Gauss linear limited 1;
}
```

Os termos de gradiente normal às superfícies, ***snGradSchemes***, são usados no cálculo dos termos laplacianos quando o teorema da divergência é aplicado. Normalmente eles são configurados da mesma forma que os termos no *laplacianSchemes*.

```
snGradSchemes
{
   default       limited 1;
}
```

Por fim, o subdicionário ***interpolationSchemes*** apresenta o método utilizado para interpolar os valores do centro de uma célula nas faces dessa célula.

A opção mais utilizada neste caso é a *linear*.

```
interpolationSchemes
{
   default       linear;
}
```

Recommended setup for most cases

```
ddtSchemes
{
    default         CrankNicolson 0.333;   //0-1
}
gradSchemes
{
    default         cellLimited Gauss linear 1;
    grad(p)         cellLimited Gauss linear 0.5;
}
divSchemes
{
    default                 none;
    div(phi,U)              Gauss linearUpwindV default;
    div(phi,omega)          Gauss linearUpwind   default;
    div(phi,k)              Gauss linearUpwind   default;
    div((nuEff*dev(T(grad(U)))))    Gauss linear;
}
laplacianSchemes
{
    default         Gauss linear limited 1;
}
interpolationSchemes
{
    default         linear;
}
snGradSchemes
{
    default         limited 1;
}
```

- **This setup is recommended for most of the cases.**
- It is very similar to the default method you will find in commercial solvers.
- In overall, this setup is second order accurate and fully bounded.
- According to the quality of your mesh, you will need to change the blending factor of the **laplacianSchemes** and **snGradSchemes** keywords.
- To keep time diffusion to a minimum, use a CFL number less than 2, and preferably below 1.
- If during the simulation the turbulence quantities become unbounded, you can safely change the discretization scheme to upwind. After all, turbulence is diffusion.
- For gradient discretization the **leastSquares** method is more accurate. But we have found that it is a little bit oscillatory in tetrahedral meshes.

265

A very stable but too diffusive numerics

```
ddtSchemes
{
    default          Euler;
}
gradSchemes
{
    default          cellLimited Gauss linear 1;
}
divSchemes
{
    default                    none;
    div(phi,U)                 Gauss upwind;
    div(phi,omega)             Gauss upwind;
    div(phi,k)                 Gauss upwind;
    div((nuEff*dev(T(grad(U)))))    Gauss linear;
}
laplacianSchemes
{
    default          Gauss linear limited 0.5;
}
interpolationSchemes
{
    default          linear;
}
snGradSchemes
{
    default          limited 0.5;
}
```

- If you are looking for extra stability, you can use this method.
- This setup is very stable but too diffusive.
- This setup is first order in space and time.
- You can use this setup to start the solution in the presence of bad quality meshes or strong discontinuities.
- Remember, you can start using a first order method and then switch to a second order method.
- According to the quality of your mesh, you will need to change the blending factor of the **laplacianSchemes** and **snGradSchemes** keywords.
- You can use this method for troubleshooting. If the solution diverges, you better check boundary conditions, physical properties, and so on.
- **Start robustly, end with accuracy.**

270

Sumário

1 Algoritmos de Acoplamento Pressão-Velocidade

2 OpenFOAM
- fvSchemes
- fvSolution
- Exemplo

No arquivo ***fvSolution*** nós vamos definir os métodos de solução dos sistemas de equações lineares (os *solvers*) e os detalhes do algoritmo de acoplamento pressão-velocidade.

Com relação aos métodos de solução dos sistemas de equações, as principais opções são

- *smoothSolvers* (método de relaxação iterativo). Principais opções: DIC, DILU, DILUGaussSeidel, GaussSeidel, symGaussSeidel.
- Gradiente Conjugado e suas variações (PCG, PBiCG, PBiCGStab, $\cdots$). Principais pré-condicionadores: diagonal, DIC, DILU, GAMG.
- Multigrid (GAMG). Principais opções de smoother: GaussSeidel e DICGaussSeidel.

Exemplo 1: *incrompressibleFluid/motorBike* .

```
solvers
{
    p
    {
        solver          GAMG;
        tolerance       1e-07;
        relTol          0.1;
        smoother        GaussSeidel;
    }

    pFinal
    {
        $p;
        tolerance       1e-6;
        relTol          0;
    }
    Phi
    {
        $p;
    }

    "(U|nuTilda)"
    {
        solver          smoothSolver;
        smoother        GaussSeidel;
        tolerance       1e-08;
        relTol          0.1;
    }
}
```

Exemplo 2: *fluid/BenardCells* .

```
solvers
    "rho.*"
    {
        solver          diagonal;
    }

    p_rgh
    {
        solver          GAMG;
        smoother        DIC;
        tolerance       1e-8;
        relTol          0.01;
    }

    p_rghFinal
    {
        $p_rgh;
        relTol          0;
    }
```

```
    "(U|e|k|epsilon)"
    {
        solver          PBiCGStab;
        preconditioner DILU;
        tolerance       1e-8;
        relTol          0.01;
    }

    "(U|e|k|epsilon)Final"
    {
        $U;
        relTol          0;
    }

}
```

Quando se trata de *solvers*, há muitas opções e combinações disponíveis no OpenFOAM.

Ao escolher os *solvers* lineares, não há uma teoria geral. Isso depende do problema e do hardware (tipo de malha, física envolvida, memória do processador, conectividade de rede, método de particionamento, entre outros).

Na maioria das vezes, o método GAMG é a melhor escolha para matrizes simétricas (por exemplo, pressão). O método GAMG deve convergir rapidamente (menos de 100 iterações). Se estiver levando mais iterações, tente alterar algumas das opções do solucionador.

Se estiver demorando muito ou for instável, use o solucionador PCG com um bom pré-condicionador. Ao executar com muitos núcleos (mais de 1000), usar o PCG pode ser uma escolha melhor.

Para matrizes assimétricas, o método PBiCGStab com o pré-condicionador DILU é uma boa escolha.

O *solver smoothSolver* com Gauss-Seidel também apresenta um desempenho muito bom.

Se o método PBiCGStab com o pré-condicionador DILU falhar misteriosamente com um erro relacionado ao pré-condicionador, utilize o *smoothSolver* ou altere o pré-condicionador.

Em geral, o solucionador PBiCGStab deve ser mais rápido do que o *smoothSolver*.

Matrizes assimétricas são montadas a partir da velocidade ($\mathbf{U}$) e das quantidades transportadas ($k, omega, epsilon, e, T$, etc.). A pressão possui uma matriz simétrica.

Geralmente, calcular a velocidade e as quantidades transportadas é barato e rápido, então é uma boa ideia usar uma tolerância rigorosa (10^{-8}) para esses campos. Resolver a pressão geralmente é a parte mais cara.

O solucionador *diagonal* é utilizado para retrosubstituição, por exemplo, ao calcular a densidade usando a equação de estado.

A configuração do algoritmo de acoplamento pressão-velocidade também é feita no *fvSolution*.

Em simulações em *steadyState*, temos que definir o subdicionário SIMPLE ou o PIMPLE.

Em simulações transientes, temos que definir o dicionário PIMPLE.

PIMPLE.

```
fvVectorMatrix UEqn
(
    fvm::ddt(U) + fvm::div(phi, U) - fvm::laplacian(nu, U)
);

solve(UEqn == -fvc::grad(p));

fvScalarMatrix pEqn
(
    fvm::laplacian(rAU, p) == fvc::div(phiHbyA)
);

U = HbyA – rAU*fvc::grad(p);
```

Momentum equation without pressure gradient term

Under-relax U Eqn

Momentum predictor

Pressure equation

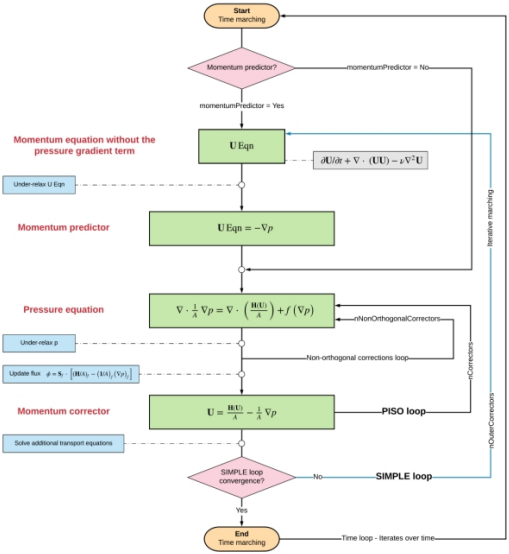
Under-relax p

Update flux $\phi = \mathbf{S}_f \cdot \left[(\mathbf{H}/\mathbf{A})_f - (1/\mathbf{A})_f (\nabla p)_f \right]$

Momentum corrector

Solve additional transport equations

PIMPLE.


Start
Time marching
Momentum predictor?
momentumPredictor = No
momentumPredictor = Yes
Momentum equation without the pressure gradient term
U Eqn
$\partial \mathbf{U}/\partial t + \nabla \cdot (\mathbf{U}\mathbf{U}) - \mu \nabla^2 \mathbf{U}$
Under-relax U Eqn
Momentum predictor
U Eqn $= -\nabla p$
Pressure equation
$\nabla \cdot \frac{1}{A} \nabla p = \nabla \cdot \left(\frac{\mathbf{H}(\mathbf{U})}{A} \right) + f\left(\nabla p\right)$
nNonOrthogonalCorrectors
Under-relax p
Non-orthogonal corrections loop
Update flux $\phi = \mathbf{S}_f \cdot \left[(\mathbf{H}(\mathbf{U})/\mathbf{A})_f - (1/\mathbf{A})_f (\nabla p)_f \right]$
nCorrectors
Momentum corrector
$\mathbf{U} = \frac{\mathbf{H}(\mathbf{U})}{A} - \frac{1}{A} \nabla p$
PISO loop
nOuterCorrectors
Solve additional transport equations
SIMPLE loop convergence?
No
SIMPLE loop
Yes
End
Time marching
Time loop - iterates over time
Iterative marching

Com relação ao *nOrthogonalCorrectors*, recomenda-se um valor de 3 para não ortogonalidade entre 70° e 85°, 2 para não ortogonalidade entre 60° e 70° e 1 para não ortogonalidade menor que 60°.

Exemplo.

```
PIMPLE
{
    nOuterCorrectors  1;
    nCorrectors       3;
    nNonOrthogonalCorrectors 1;
}
```

Exemplo.

```
SIMPLE
{
    nNonOrthogonalCorrectors 0;
    consistent    yes;
    residualControl
    {
        p               1e-2;
        U               1e-3;
        "(k|epsilon|omega|f|v2)" 1e-3;
    }
}

relaxationFactors
{
    equations
    {
        U               0.9; // 0.9 is more stable but 0.95 more convergent
        ".*"            0.9; // 0.9 is more stable but 0.95 more convergent
    }
}
```

Sumário

1 Algoritmos de Acoplamento Pressão-Velocidade

2 OpenFOAM
- fvSchemes
- fvSolution
- Exemplo

Para entender na prática como essas configurações funcionam, vamos analisar um caso bem simples: transporte advectivo e difusivo de uma propriedade escalar em uma dimensão.

Abra o caso *temperatureScalarTransport*. Para rodar este caso, execute os comandos *blockMesh*, *setFields* e *foamRun*, nessa ordem.

Rode diferentes simulações com diferentes parâmetros e diferentes configurações (modifique as configurações nos arquivos *fvSchemes* e *fvSolution*).

Veja quais são as melhores combinações para este caso. Observe os casos em que a simulação diverge. Observe também os casos em que aparecem oscilações indesejadas na simulação.

Por fim, dê uma olhada com calma em diferentes tutoriais do OpenFOAM, focando nos arquivos *fvSchemes* e *fvSolution*.